Práctica 4: Coach Financiero Generativo con Agentes LangGraph

Autores:

Diego Esclarín (51729611N)
Sofía Contreras (09848471D)

Aplicaciones de Inteligencia Artificial
Grado en Ingeniería en Inteligencia Artificial
Universidad Rey Juan Carlos
Curso 2025–26

Índice

1. Introducción **3**
1.1. Objetivos del Proyecto 3
1.2. Alcance y Limitaciones 3

2. Contexto y Motivación **5**
2.1. Problemática Actual 5
2.2. Fundamentos Tecnológicos 5

3. Estado del Arte **7**
3.1. Sistemas Tradicionales de Gestión Financiera 7
3.2. Técnicas de Procesamiento de Lenguaje Natural en Finanzas 7
3.3. Agentes Conversacionales en Dominios Especializados 7
3.4. Arquitecturas de Agentes Basados en Grafos 7

4. Propuesta y Desarrollo **9**
4.1. Arquitectura del Sistema 9
4.2. Agente LangGraph 9
4.3. Coach Clásico 9
4.4. Motor de Análisis Financiero 9
4.5. Sistema de Memoria Persistente 10
4.6. Implementación Técnica 10

5. Conclusiones y Trabajo Futuro **11**
5.1. Trabajo Futuro 11

1. Introducción

El propósito fundamental de este proyecto es el desarrollo de un sistema de asesoramiento financiero personal basado en agentes de inteligencia artificial generativa. En un entorno donde el volumen de transacciones digitales crece exponencialmente [1], la capacidad de analizar datos financieros de manera inteligente deja de ser un lujo para convertirse en una necesidad técnica. El sistema diseñado actúa como un coach financiero activo capaz de comprender consultas en lenguaje natural, analizar patrones de gasto y proporcionar recomendaciones personalizadas.

El núcleo de la investigación se centra en la implementación de un agente inteligente utilizando la arquitectura LangGraph [2], que permite al sistema tomar decisiones dinámicas sobre qué información consultar y cómo responder a las consultas del usuario. A diferencia de los sistemas basados en reglas estáticas [3], el agente desarrollado posee la capacidad de razonar sobre los datos financieros y mantener un estado persistente entre sesiones.

Finalmente, el proyecto destaca por su capacidad de adaptación continua mediante mecanismos de memoria persistente [4]. Esta característica permite al sistema evolucionar con el usuario, recordando objetivos financieros y generando alertas contextualizadas, lo que lo posiciona como una solución innovadora en el campo de la tecnología financiera personal [5].

1.1. Objetivos del Proyecto

Los objetivos principales de esta práctica se estructuran en tres niveles complementarios:

Objetivo técnico: Diseñar e implementar un agente conversacional basado en LangGraph capaz de analizar datos financieros personales utilizando un conjunto de doce herramientas especializadas. El sistema debe demostrar capacidad de razonamiento multi-paso, encadenamiento de herramientas y mantenimiento de estado persistente.

Objetivo funcional: Desarrollar una aplicación funcional que permita a los usuarios establecer objetivos de ahorro, recibir alertas personalizadas, consultar análisis detallados de sus gastos y obtener recomendaciones financieras contextualizadas basadas en su historial de transacciones real.

1.2. Alcance y Limitaciones

El proyecto se centra exclusivamente en el análisis de datos financieros personales estructurados, utilizando como fuente principal un conjunto de transacciones bancarias en formato CSV.

Las limitaciones principales del sistema incluyen: la dependencia de la calidad y completitud de los datos de entrada o la incapacidad de procesar información financiera no estructurada (como facturas o documentos PDF). Además, el sistema no realiza predicciones financieras complejas ni ofrece recomendaciones de inversión, centrándose exclusivamente en el análisis de gastos y consejos de ahorro basados en patrones históricos.

2. Contexto y Motivación

La gestión financiera personal representa uno de los desafíos más significativos hoy en día. El volumen de transacciones digitales crece exponencialmente [1], mientras que los usuarios se enfrentan diariamente a la necesidad de comprender sus patrones de gasto y establecer objetivos de ahorro realistas. Sin embargo, las herramientas disponibles tradicionalmente ofrecen o bien análisis retrospectivo básico o requieren conocimientos técnicos avanzados para su configuración y uso efectivo.

2.1. Problemática Actual

El panorama actual de las aplicaciones de gestión financiera personal presenta limitaciones significativas. Las soluciones comerciales suelen ofrecer dashboards genéricos que presentan datos de forma visualmente atractiva pero carecen de capacidad de interpretación contextual [3]. Estas herramientas pueden mostrar que un usuario ha gastado un 20 % más en restaurantes, pero son incapaces de explicar por qué ha ocurrido este aumento o cómo podría optimizarse.

Por otro lado, las soluciones basadas en hojas de cálculo requieren un alto grado de conocimiento técnico. Esta barrera limita la adopción de prácticas financieras saludables y perpetúa la toma de decisiones basada en intuición en lugar de en análisis de datos objetivos [5].

2.2. Fundamentos Tecnológicos

La reciente evolución de los modelos de lenguaje grandes (LLMs) y las arquitecturas de agentes basados en grafos ha abierto nuevas posibilidades en el campo de la asistencia financiera personal [6]. Estos sistemas combinan la capacidad de comprensión del lenguaje natural con el razonamiento estructurado, permitiendo crear interfaces conversacionales que pueden entender consultas complejas y realizar análisis multi-paso.

La arquitectura LangGraph [2] ofrece un marco ideal para este tipo de aplicaciones al permitir el diseño de agentes que pueden decidir dinámicamente qué herramientas utilizar, encadenar múltiples operaciones y mantener estado entre interacciones. A diferencia de los sistemas basados en pipelines fijos, los agentes LangGraph pueden adaptar su comportamiento a las necesidades específicas de cada consulta.

3. Estado del Arte

El campo de la tecnología financiera personal (FinTech) ha evolucionado significativamente en la última década [1]. Esta sección analiza las principales aproximaciones tecnológicas existentes y posiciona nuestra propuesta dentro del panorama actual.

3.1. Sistemas Tradicionales de Gestión Financiera

Las aplicaciones convencionales se clasifican en tres categorías principales. Las hojas de cálculo y software de contabilidad ofrecen flexibilidad pero requieren intervención manual constante [5]. Las aplicaciones de agregación bancaria como Mint o YNAB automatizan la importación de datos pero mantienen un enfoque reactivo, limitándose a presentar información histórica sin capacidad de interpretación contextual [3]. Los robo-advisors automatizan inversiones pero su ámbito se restringe exclusivamente a la gestión de patrimonios.

3.2. Técnicas de Procesamiento de Lenguaje Natural en Finanzas

La aplicación de PLN al dominio financiero ha abierto nuevas posibilidades [7]. Los primeros sistemas utilizaron técnicas de clasificación supervisada como Naive Bayes y SVM, logrando precisiones aceptables (70-85 %) pero con rigidez ante nuevos patrones. El análisis de sentimiento ha permitido detectar estrés financiero, aunque enfrenta desafíos por la subjetividad del lenguaje. Las técnicas de NER con modelos basados en transformers como BERT han mejorado la precisión en extracción de entidades, pero requieren grandes volúmenes de datos etiquetados.

3.3. Agentes Conversacionales en Dominios Especializados

Los recientes avances en LLMs han impulsado el desarrollo de agentes conversacionales especializados [6]. Los primeros sistemas utilizaban árboles de decisión y reglas predefinidas, demostrando ser frágiles ante variaciones en el lenguaje. Las arquitecturas seq-to-seq y modelos basados en transformers permitieron generar respuestas más naturales pero seguían limitados por su naturaleza sin estado.

La generación más reciente de agentes con herramientas (tool-using agents) representa un avance fundamental [8]. Sistemas como AutoGPT y los agentes LangGraph permiten que los modelos de lenguaje no solo procesen texto, sino que también ejecuten acciones y realicen análisis complejos.

3.4. Arquitecturas de Agentes Basados en Grafos

LangGraph emerge como una de las arquitecturas más prometedoras para el desarrollo de agentes complejos [2]. A diferencia de los sistemas sin estado, LangGraph permite mantener información contextual entre múltiples interacciones, fundamental para asistentes financieros personales [4]. La arquitectura de grafos permite encadenar múltiples operaciones de razonamiento y seleccionar dinámicamente qué herramientas utilizar basándose en el contexto específico de cada consulta.

4. Propuesta y Desarrollo

La propuesta técnica se fundamenta en el diseño de un agente conversacional inteligente especializado en asesoramiento financiero personal. El sistema combina procesamiento de lenguaje natural con análisis financiero estructurado, implementando una arquitectura dual que permite comparar un agente LangGraph frente a una implementación clásica basada en clasificación de intenciones.

4.1. Arquitectura del Sistema

El sistema sigue una arquitectura modular con cuatro capas principales. La capa de presentación utiliza Streamlit para proporcionar una interfaz web interactiva con métricas financieras y chat conversacional. La capa de agentes contiene dos implementaciones paralelas: el agente LangGraph con grafo de estado y el coach clásico con clasificador de intenciones. La capa de analytics implementa el motor de análisis financiero con doce funciones especializadas. La capa de datos gestiona la carga y procesamiento de transacciones mediante pandas y JSON para persistencia.

4.2. Agente LangGraph

El componente central implementa un grafo con seis nodos: load_memory carga el estado persistente, summarize gestiona el resumen automático de conversaciones largas, agent invoca al LLM con acceso a doce herramientas, tools ejecuta las herramientas seleccionadas, check_goals evalúa objetivos de ahorro, y save_memory persiste el estado actualizado. El agente puede encadenar múltiples herramientas mediante bucle ReAct [9].

4.3. Coach Clásico

La implementación clásica sirve como línea base para evaluar las mejoras introducidas por la arquitectura LangGraph. Utiliza un clasificador basado en keywords que categoriza las consultas en cuatro intenciones: savings_advice, concept_explanation, spending_query y general_chat. Para cada intención, el sistema ensambla un contexto fijo con datos preseleccionados y plantillas de respuesta específicas.

4.4. Motor de Análisis Financiero

El motor de analytics procesa aproximadamente 900 transacciones mensuales mediante pandas. Implementa trece funciones especializadas que incluyen resúmenes mensuales, análisis de tendencias, desglose por categoría, detección de anomalías, gestión de gastos recurrentes y análisis de inversiones. El sistema utiliza técnicas estadísticas basadas en desviación estándar y percentiles para identificar patrones anómalos y generar alertas personalizadas. Adicionalmente, el módulo de inversiones analiza automáticamente diferentes instrumentos financieros como Letras del Tesoro, acciones Apple, S&P 500, Bitcoin, planes de pensiones y fondos de inversión, proporcionando un resumen completo del portafolio de inversión del usuario.

Las trece herramientas disponibles para el agente LangGraph se dividen en dos categorías: nueve herramientas de análisis financiero (monthly_summary, spending_trends, category_breakdown, savings_rate, anomalies, recurring_expenses,

top_expenses, search_by_description, investment_summary) y cuatro herramientas de gestión de objetivos (set_goal, check_goal_progress, list_goals, remove_goal).

4.5. Sistema de Memoria Persistente

La memoria persistente permite al sistema mantener estado entre sesiones. Los usuarios pueden establecer objetivos de gasto máximo mensual por categoría, que se almacenan en archivos JSON individuales. El sistema monitorea continuamente el progreso hacia estos objetivos y genera alertas automáticas cuando el gasto actual supera el 80 % del límite establecido.

Adicionalmente, el sistema implementa un mecanismo de resumen automático de conversaciones que se activa cuando el historial supera un umbral predefinido. Un LLM dedicado genera resúmenes concisos que capturan temas tratados, datos financieros relevantes y decisiones tomadas. Estos resúmenes se persisten en memoria y permiten mantener conversaciones largas sin exceder la ventana de contexto del modelo principal.

4.6. Implementación Técnica

El proyecto se implementa utilizando Python 3.12 con LangGraph [2], Groq API para los modelos de lenguaje [10], Streamlit para la interfaz [11] y pandas para el análisis de datos [12]. La aplicación soporta cuatro modelos LLM diferentes: Llama 3.3 70B (modelo por defecto), Llama 3.1 8B, Mixtral 8x7B y Gemma 2 9B, permitiendo al usuario seleccionar el balance óptimo entre calidad de respuesta y velocidad de procesamiento según sus necesidades.

5. Conclusiones y Trabajo Futuro

El proyecto tiene implicaciones significativas tanto a nivel técnico como social. Demuestra la viabilidad de utilizar arquitecturas de agentes avanzados en aplicaciones prácticas del mundo real, superando las limitaciones de los chatbots tradicionales. Las herramientas de asesoramiento financiero inteligente pueden ayudar a democratizar el acceso a consejos financieros personalizados, tradicionalmente reservados para personas con altos ingresos o conocimientos especializados.

En conclusión, el Coach Financiero Generativo representa no solo un éxito técnico en la implementación de agentes inteligentes, sino también una contribución significativa al campo de la tecnología financiera personal, estableciendo un nuevo estándar para las aplicaciones de asesoramiento financiero basadas en IA.

5.1. Trabajo Futuro

Basándonos en los resultados y limitaciones identificadas, proponemos varias líneas de desarrollo futuro. Una de las más prometedoras es la integración con las prácticas anteriores del curso, creando un ecosistema financiero personal completo. La incorporación de capacidades predictivas mediante los modelos de machine learning desarrollados en prácticas anteriores podría anticipar tendencias de gasto futuras y recomendar estrategias de ahorro proactivas.

La extensión del sistema para integrar las herramientas de análisis de datos de la práctica 2 y las capacidades de visualización de la práctica 3 permitiría crear una plataforma unificada. La integración con APIs bancarias directas podría eliminar la necesidad de importación manual de datos.

Referencias Bibliográficas

[1] M. Johnson y S. Smith, «The Evolution of Personal Finance Management Applications,» *Journal of Financial Technology*, vol. 15, n.º 3, págs. 45-67, 2023.

[2] L. Team, «LangGraph: Building Stateful, Multi-Actor Applications with LLMs,» *LangChain Documentation*, 2024. dirección: `https://python.langchain.com/docs/langgraph`

[3] W. Chen y R. Kumar, «Conversational AI in Financial Services: Opportunities and Challenges,» *International Journal of AI in Finance*, vol. 8, n.º 2, págs. 112-134, 2023.

[4] A. Wang y S. Li, «Memory Systems for Conversational Agents: Design and Evaluation,» *Proceedings of ACL 2023*, págs. 234-248, 2023.

[5] M. Garcia y C. Rodriguez, «AI-Powered Personal Finance Management: A Comprehensive Survey,» *ACM Computing Surveys*, vol. 56, n.º 4, págs. 1-35, 2024.

[6] T. Schick, L. Schmid, Y. Dwivedi, S. Mishra et al., «Tool-Using Large Language Models: A Survey,» *arXiv preprint arXiv:2304.08354*, 2023.

[7] L. Zhang y D. Brown, «Natural Language Processing in Financial Applications: A Review,» *Expert Systems with Applications*, vol. 214, pág. 119 234, 2023.

[8] L. Team, «LangChain: A Framework for Building Language Model Applications,» *arXiv preprint arXiv:2309.03528*, 2023. dirección: `https://github.com/langchain-ai/langchain`

[9] S. Yao, J. Zhao, Y. Dossantos, T. Yu, N. Du, K. Narasimhan et al., «ReAct: Synergizing Reasoning and Acting in Language Models,» *arXiv preprint arXiv:2210.03629*, 2022.

[10] G. Team, «Groq: Fast Inference for Large Language Models,» *Groq Documentation*, 2024. dirección: `https://console.groq.com/docs`

[11] S. Team, «Streamlit: An Open-Source Framework for Machine Learning Applications,» *Streamlit Documentation*, 2021. dirección: `https://docs.streamlit.io`

[12] W. McKinney et al., «Pandas: A Python Data Analysis Library,» *PyData*, 2020. dirección: `https://pandas.pydata.org`